1.Aim of the practical: **(A)** Implement linear regression using Python by calculating the slope (a) and intercept (b) manually through formulas and plot the best fit line for a given dataset using Matplotlib.

(B) Build a simple linear regression model that predicts output values based on a single input feature using Python and machine learning libraries.

2.Objective: To generate a dataset with random noise using NumPy.

- To apply linear regression formulas to calculate the slope and intercept.
- To apply linear regression for solving real-world prediction problems.
- To understand how to fit a model using training data and make predictions on unseen data.

3.Algorithm/Flowchart:

(A):

```
import numpy as np
import pandas as pd
x=np.linspace(1,6,10)
```

```
x
```

```
array([1.        , 1.55555556, 2.11111111, 2.66666667, 3.22222222,
       3.77777778, 4.33333333, 4.88888889, 5.44444444, 6.        ])
```

```
temp=[]
for n in x:
    t=(np.random.choice(range(1,3)))
    temp.append(n+t)
    y = np.array(temp)
y
```

```
array([3.        , 3.55555556, 3.11111111, 3.66666667, 4.22222222,
       4.77777778, 6.33333333, 5.88888889, 6.44444444, 7.        ])
```

```
import matplotlib.pyplot as plt
```

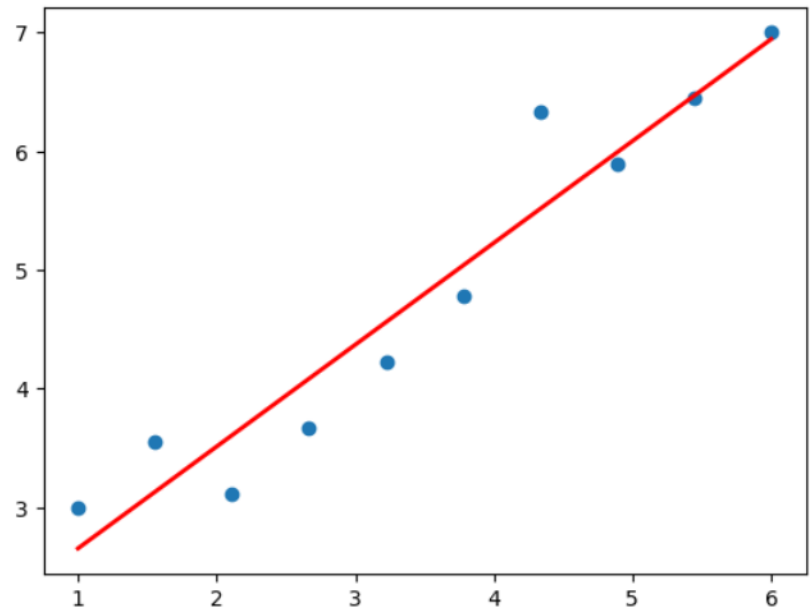
```
m, c = np.polyfit(x, y, 1)


best_fit_y = m * x + c

plt.scatter(x,y)
plt.plot(x, best_fit_y, color='red', linewidth=2, label=f'Best Fit Line: y = {m:.2f}x + {c:.2f}')
plt.show()
```

(B):

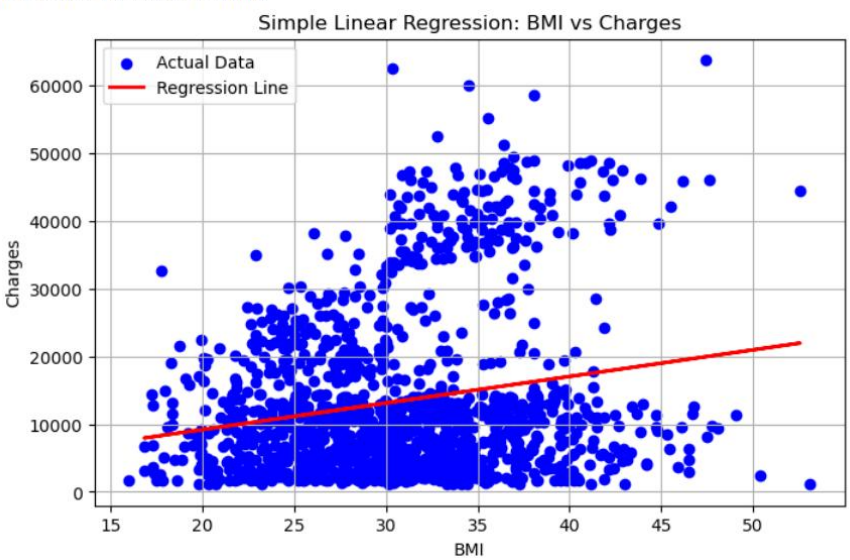
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score


data = pd.read_csv('insurance.csv')
X = data[['bmi']].values  # Input feature
y = data['charges'].values  # Target variable

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("\n📊 Simple Linear Regression Results (BMI vs Charges)")
print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)
print("Mean Squared Error:", mse)
print("R^2 Score:", r2)
```

```
plt.figure(figsize=(8,5))
plt.scatter(X, y, color='blue', label='Actual Data')
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Regression Line')
plt.title("Simple Linear Regression: BMI vs Charges")
plt.xlabel("BMI")
plt.ylabel("Charges")
plt.legend()
plt.grid(True)
plt.show()
```

4.Result/Output:




```
📊 Simple Linear Regression Results (BMI vs Charges)
Slope: 392.43654416987977
Intercept: 1353.0730722046683
Mean Squared Error: 149085057.03839505
R^2 Score: 0.03970193117941878
```



Learning outcomes (What I have learnt):

- Able to implement a basic machine learning model using Python.
- Understand the working of linear regression with one independent and one dependent variable.
- Learn how to split data and fit a model using scikit-learn.